NESTED GREEDY SEARCH FOR TOOL SELECTION

Anonymous authors
Paper under double-blind review

ABSTRACT

1 PRELIMINARY

1.1 FORMULATION

Tool selection for LLMs is essentially a complex combinatorial optimization problem. When the candidate tool library is vast, exhaustively evaluating all tool combinations is impractical (NP-hard). While standard greedy algorithms are efficient, they often fail to account for the complex interactions between tools.

1.2 LIMITATION OF NGA

Algorithm 1 Nested Greedy Algorithm (NGA)

Input: Ground set V; budget b; solution-set size k
Output: Solution set S

$S \leftarrow \emptyset$
for $j = 1$ ... k **do**
  $X_j \leftarrow \emptyset$
  for $t = 1$ ... b **do**
    $v^* \leftarrow \arg\max_{v \in V \setminus X_j} \Delta_{ps}(v \mid X_j)$
    $X_j \leftarrow X_j \cup \{v^*\}$
  end for
  $S \leftarrow S \cup X_j$
end forreturn S

In LLM tool selection and complex reasoning tasks, the traditional Nested Greedy Algorithm (NGA)—while theoretically overcoming the constraints of strict submodularity—exhibits significant drawbacks in practical engineering deployment. The Evolutionary Nested Greedy Algorithm (ENGA) was introduced specifically to address these pain points.

Limitations of NGA Explosive Computational Overhead (Computational Intractability): NGA's inner loop requires frequent calculation of the marginal gain Δ. Applying NGA directly to LLM tool selection implies that evaluating each candidate tool necessitates a full rollout or forward inference pass using the large model. For a vast tool library $\mathcal{V}$, the resulting $O(|\mathcal{V}|^2)$ cost in terms of LLM API calls is unacceptable regarding both time and financial expense.

Myopic Heuristic Bias: NGA relies on a predefined, static objective function $h(S)$. In complex agentic scenarios, the "quality" of a tool is multidimensional (encompassing relevance, mutual exclusivity, latency, and token costs). It is difficult for human experts to arbitrarily determine a perfect, static weighting ratio. If the initial objective function is poorly defined, NGA will steadfastly converge to a suboptimal solution.

Vulnerability to Local Optima: Although NGA possesses greater "lookahead" capability than standard greedy algorithms, it fundamentally follows a deterministic greedy path. Once an erroneous choice is made early in the outer loop (e.g., selecting a "poison" tool that appears relevant but fails to synergize with others later), NGA lacks a backtracking mechanism and cannot escape the trap of local optima.

ENGA (Advantages of ENGA) Drastic Reduction in Computational Complexity (Decoupling Inference from Search): ENGA completely decouples expensive large-model inference (fitness evaluation) from the inner-loop construction of the candidate set. The inner loop utilizes a lightweight proxy function $\tilde{\Delta}$—controlled by hyperparameters $\boldsymbol{\alpha}$—to perform ultra-fast computations (involving only vector dot products and scalar operations).

The frequency of large model invocations has been reduced from the "tool-level probing" seen in NGA to the "population-level evaluation" of ENGA, significantly lowering system latency.

Adaptive Multi-Objective Tuning: ENGA delegates the challenge of balancing gains across different dimensions to an evolutionary algorithm. By abstracting semantic relevance ($\mathcal{I}$), tool synergy ($\mathcal{S}$), and system execution cost ($\mathcal{C}$) into a weight vector ($\boldsymbol{\alpha}$), the system dynamically evolves the optimal weight distribution for specific tasks, eliminating the need for manual hyperparameter tuning.

Global Exploration via Population Dynamics: Evolutionary algorithms (such as CMA-ES or genetic algorithms) inherently possess global optimization capabilities. By maintaining a diverse population of parameters and introducing stochastic perturbations through crossover and mutation, ENGA explores a broader solution space, effectively avoiding the premature convergence issues that plague deterministic greedy algorithms in complex, non-convex spaces.

Fit for Black-box LLMs: LLM output generation, hallucination probability, and actual task success rates are typically non-differentiable. ENGA bypasses gradient-based optimization; it requires only a final scalar reward (fitness) from the large model to drive the iterative refinement of tool combination logic, making it ideally suited to the engineering paradigms of current agentic systems.

1.3 EVOLUTIONARY NGA

By applying multi-objective parameterization (introducing α_1, α_2) to the computationally expensive marginal gain Δ within the NGA inner loop—and solving it using Evolutionary Algorithms (EA)—a hybrid optimization architecture is effectively constructed.

In the complex environment of LLM tool use, this design elegantly bypasses the non-differentiable black-box nature of large language models.

novel ENGA :

1. Parameterized deconstruction of the marginal gain Δ

The original NGA required an actual evaluation of $\Delta_{p_S}(v|X_j)$ within the inner loop; in an agent scenario, this implies an extremely time-consuming full inference process. By introducing weight parameters α, the "true marginal gain" can be approximated via dimensionality reduction into a low-cost proxy heuristic function:

$$\tilde{\Delta}(v|X_j) = \alpha_1 \cdot \phi_{\text{utility}}(v) + \alpha_2 \cdot \phi_{\text{synergy}}(v, X_j) - \alpha_3 \cdot \phi_{\text{cost}}(v)$$

$\phi_{\text{utility}}(v)$: The intrinsic utility of tool v (e.g., a prior score based on historical invocation success rates).

$\phi_{\text{synergy}}(v, X_j)$: The synergy or diversity between tool v and the currently selected tool set X_j (e.g., calculated via the orthogonality of tool embeddings).

$\phi_{\text{cost}}(v)$: The token consumption or latency associated with invoking the tool.

Under this framework, the greedy selection step in the NGA inner loop no longer relies on model rollouts but instead transforms into a rapid scalar computation.

2. Two Approaches to Using Evolutionary Algorithms (EA)

There are two highly suitable paths for using evolutionary algorithms to "solve" this system:

Path A: Evolving Hyperparameters (Evolving the Heuristics) In this scenario, an "individual" in the evolutionary algorithm is a weight vector $\boldsymbol{\alpha} = [\alpha_1, \alpha_2, \dots]$.

1. Initialize Population: Randomly generate multiple sets of $\boldsymbol{\alpha}$.

2. Rapid Evaluation (NGA as Decoder): For each set of $\boldsymbol{\alpha}$ in the population, plug it into the inner function $\tilde{\Delta}$, quickly execute the NGA algorithm, and output a final tool combination $S_{\boldsymbol{\alpha}}^{\text{NGA}}$.

3. Calculate Fitness: Feed the tool set $S_{\boldsymbol{\alpha}}^{\text{NGA}}$ to the LLM and execute it on a validation set; the resulting actual task success rate (or composite reward) serves as the fitness for that $\boldsymbol{\alpha}$.

4. Mutation and Crossover: Discard low-scoring $\boldsymbol{\alpha}$ values and generate new weights through crossover and mutation until convergence.

Advantages: This method leverages the powerful optimization capabilities of algorithms like CMA-ES (Covariance Matrix Adaptation Evolution Strategy) in continuous spaces while significantly reducing the number of LLM calls.

Path B: Multi-Objective Evolutionary Optimization (MOEA) If you prefer not to force a weighted sum of objectives across different dimensions (such as accuracy and latency), you can treat α_1 and α_2 as two independent optimization objectives (e.g., $f_1 =$ accuracy, $f_2 =$ tool set redundancy).

By using multi-objective evolutionary algorithms like NSGA-II, you can directly derive a set of Pareto optimal tool combinations. Advantages: During deployment, you can dynamically select different tool combinations along the Pareto frontier based on current system load (e.g., choosing a combination with extremely high accuracy—but involving more tools and longer execution times—during off-peak periods, and switching to a low-latency combination during peak periods).

Why is this approach highly feasible? Avoids the issue of missing gradients: LLM tool selection is inherently a discrete combinatorial optimization problem (a variant of the 0/1 knapsack problem); gradient-based deep reinforcement learning methods (such as PPO) often struggle here with sparse rewards and an exploding action space. Evolutionary algorithms (EA), by contrast, are purely gradient-free global optimizers.

Breaks out of local optima: Traditional greedy algorithms are prone to making errors at the initial step. By using EA to continuously perturb α—or even directly crossover and recombine tool sets S—the algorithm gains the "escape capability" needed to break free from local optima.

1.4 SCENARIOS

In LLMs and agents, tool selection/function calling serves as the core mechanism enabling models to transcend the limitations of **pure text generation** and engage in substantive interaction with external systems.

There are key scenarios:

1. Information Retrieval & Knowledge Augmentation (RAG) Single-step / Static Selection/One-shot decision

Key idea: Query -(Tool selection) Tool -(excute) Response

The system simply needs to identify the top-k most relevant tools from the tool library based on the user's current query. There are no causal chains linking the tools, nor is there any dynamic feedback—such as a change in page or system State following the execution of Tool A that subsequently unlocks Tool B.

2. Data Analysis & Execution Hybrid / Semi-dynamic / depends on task difficulty

Static Mode (Simple Query): A user asks, "Which product had the highest sales last month?" The LLM selects the SQL tool, generates an SQL Statement, and retrieves the result. This is a classic static, single-step process.

Dynamic Mode (Iterative Analysis / Code Interpreter):

The LLM selects a Python tool to execute a data processing script.

The code execution fails (generating a new State, s_t.

The LLM captures the error message and dynamically evaluates the next step: should it switch to a plotting tool, rewrite the code, or use a data-cleaning API?

3. Multi-Step Agentic Workflows Multi-step/ Sequential Decision Making

Key idea: s_0-(a_0) - s_1(a_1) – s_2 (a_2) ... s_n

e.g., In scenarios such as web navigation (e.g., WebArena Zhou et al. (2024)), complex cross-service workflows, and agentic planning with large models, tasks are explicitly modeled as Partially Observable Markov Decision Processes (POMDPs).

State-dependent: The selection and execution of a tool a_t in the previous step (e.g., clicking "Search Flights" on a booking site) transitions the environment State to s_{t+1} (e.g., arriving at the flight list page). The set of tools available—or appropriate—for the next step depends entirely on this new State, s_{t+1}.

Sequential Dependency: The selection of tools involves strict pre-conditions and post-conditions that cannot be fully anticipated upfront based solely on the initial query.

Table 1: Summarization of 3 Scenarios

Dimension	Static Tool Selection	Dynamic / Sequential Tool Selection
Primary Mapping Scenarios	Scenarios 1 & simple Scenario 2	Scenario 3 and complex Scenario 2 Web navigation, etc.)
Core Decision Formula	$v^* = \arg\max_v \text{Relevance}(v, q)$	$v_t^* = \arg\max\_v \tilde{\Delta}(v \mid \mathcal{X}_{<t}, s_t)$
Tool Interaction	Independent or parallel selection (no causality)	Strong dependency, sequential nature, collaboration or conflict
Environmental Feedback	No feedback or single result return only	Real-time environmental state changes (DOM tree updates, API state changes)
Optimization Focus	Semantic relevance, recall	State alignment, temporal coherence, execution risk management

1.5 SUMMARIZATION

2 METHOD

2.1 NGA

2.2 STATIC ENGA

Objective

$$\tilde{\Delta}(v|X_j) = \alpha_1 \cdot \mathcal{I}_{\text{rel}}(v, q) + \alpha_2 \cdot \mathcal{S}_{\text{syn}}(v, \mathcal{X}_j) - \alpha_3 \cdot \mathcal{C}_{\text{run}}(v) \tag{1}$$

1. α_1: Semantic Relevance – Focusing on Tool Representation

This parameter measures the degree of direct matching between a candidate tool v and the current user query q or the current RAG/QA context.

Physical Meaning: Initial alignment of task intent (Intent Alignment).

Calculation Method: Dense vector retrieval techniques can be employed. By embedding the tool's description (Tool Description/Schema) and the current user query, $\mathcal{I}_{\text{rel}}$ is defined as the cosine similarity between the two in high-dimensional space.

2. α_2: Synergy & Information Gain – Focusing on Structured Representation

In the workflow of a RAG/QA agent, tools often exhibit strong complementarity. If a "content-based retrieval tool" has already been selected, choosing another highly similar tool should result in significantly diminished marginal utility.

Physical Meaning: Non-redundancy or mutual information of the tool combination.

Calculation Method:

Orthogonality Penalty: Calculate the similarity between the candidate tool v's embedding and the embeddings of all tools in the currently selected set X_j; use the maximum value as a penalty term (or conversely, define it as a diversity gain).

Schema-based Mutual Information: Evaluate whether the output data structure of tool v can serve as input for a tool within X_j. Particularly when chaining tools via the Model Context Protocol (MCP), if the output of tool A aligns precisely with the input parameter specifications of tool B, this topological connectivity can yield a very high $\mathcal{S}_{\text{syn}}$ score.

3. α_3: Runtime Penalty – Focusing on Feedback Metrics

In production deployments, the Large Language Model's (LLM) context window and response latency act as hard constraints.

Physical Meaning: The "friction cost" associated with system execution.

Calculation Method:

Token Consumption: The length of the tool schema description and the average number of tokens in the expected return result.

Time Latency: The average latency of historical calls to the specific API or tool.

Failure Rate: The probability—calculated from historical logs—that the tool triggers LLM hallucinations or execution errors.

Acts as a negative constraint to prevent the EA from evolving a bloated toolchain—containing a dozen or more redundant tools—that causes severe system timeouts or exceeds token limits.

Within this three-dimensional framework, the Evolutionary Algorithm (EA) optimizes the vector $\boldsymbol{\alpha} = [\alpha_1, \alpha_2, \alpha_3]$.

The optimal distribution of $\boldsymbol{\alpha}$ varies depending on the RAG/QA scenario. For example:

For simple repeat-purchase queries, the EA might evolve a weight combination with a very high α_3 (prioritizing low latency). For highly ambiguous, exploratory RAG/QA tasks (e.g., "Help me plan a complete summer camping gear list"), the EA would evolve a weight combination with a very high α_2 (encouraging high synergy and diversity among tools).

Key Idea The outer layer employs an evolutionary algorithm (EA) to evolve the weight vector α, while the inner layer utilizes a nested greedy algorithm (NGA) combined with a surrogate heuristic function to rapidly construct tool subsets Finally, an LLM performs the actual fitness evaluation.

Algorithm 2 Evolutionary Nested Greedy Algorithm (ENGA) for General Tool Selection

Input: Candidate tool library $\mathcal{V}$; Outer budget k; Inner budget b; User query q; Population size P; Max generations G
Output: Optimal tool subset $\mathcal{S}^*$

Initialize population $\mathcal{P} \leftarrow \{\boldsymbol{\alpha}^{(1)}, \boldsymbol{\alpha}^{(2)}, \ldots, \boldsymbol{\alpha}^{(P)}\}$, where $\boldsymbol{\alpha} = [\alpha_1, \alpha_2, \alpha_3]$
Global best fitness $F^* \leftarrow -\infty$
Global best tool subset $\mathcal{S}^* \leftarrow \emptyset$
for generation $g = 1 \ldots G$ **do**
  for each individual $\boldsymbol{\alpha} \in \mathcal{P}$ **do**
    // Phase 1: Fast NGA Decoding using Proxy Heuristic
    $\mathcal{S}_{\boldsymbol{\alpha}} \leftarrow \emptyset$
    for $j = 1 \ldots k$ **do**
      $\mathcal{X}_j \leftarrow \emptyset$ ▷ Initialize inner tool bundle
      for $t = 1 \ldots b$ **do**
        // Calculate proxy marginal gain $\tilde{\Delta}$
        $v^* \leftarrow \arg\max_{v \in \mathcal{V} \setminus \mathcal{X}_j} \left(\alpha_1 \cdot \mathcal{I}_{\text{rel}}(v, q) + \alpha_2 \cdot \mathcal{S}_{\text{syn}}(v, \mathcal{X}_j) - \alpha_3 \cdot \mathcal{C}_{\text{run}}(v) \right)$
        $\mathcal{X}_j \leftarrow \mathcal{X}_j \cup \{v^*\}$
      end for
      $\mathcal{S}_{\boldsymbol{\alpha}} \leftarrow \mathcal{S}_{\boldsymbol{\alpha}} \cup \mathcal{X}_j$
    end for
    // Phase 2: Fitness Evaluation via LLM Execution
    $F(\boldsymbol{\alpha}) \leftarrow \text{EvaluateFitness}(\mathcal{S}_{\boldsymbol{\alpha}}, q)$ ▷ Execute constructed toolset and get reward
    if $F(\boldsymbol{\alpha}) > F^*$ **then**
      $F^* \leftarrow F(\boldsymbol{\alpha})$
      $\mathcal{S}^* \leftarrow \mathcal{S}_{\boldsymbol{\alpha}}$
    end if
  end for
  // Phase 3: Evolutionary Operations (e.g., CMA-ES or Genetic Algorithm)
  $\mathcal{P}_{\text{parents}} \leftarrow \text{Selection}(\mathcal{P}, F)$ ▷ Select elites based on fitness
  $\mathcal{P} \leftarrow \text{CrossoverAndMutation}(\mathcal{P}_{\text{parents}})$ ▷ Generate new weights For next generation
end for
  return $\mathcal{S}^*$

Explanation **1. Decoupling of Input Parameters**: The algorithm abstracts away specific business logic; inputs consist solely of the tool library $\mathcal{V}$, budget constraints (k, b), and a general query q.

2. Phase 1 (Rapid Decoding): This step involves purely lightweight mathematical operations (such as vector dot products and similarity retrieval). Using the weights $\boldsymbol{\alpha}$ from the current generation, a candidate set $\mathcal{S}_{\boldsymbol{\alpha}}$ containing $k \times b$ tools is rapidly constructed. This avoids invoking the Large Language Model (LLM) within the inner loop.

3. Phase 3 (Real Evaluation): Perform a single "real" 'EvaluateFitness' assessment on the fully constructed toolset $\mathcal{S}_{\boldsymbol{\alpha}}$ (e.g., by feeding it to an LLM for execution and calculating metrics such as task success rate and composite reward), using the result as the fitness score for the current hyperparameters $\boldsymbol{\alpha}$.

4. Phase 4 (Population Evolution): Drive the evolution of the weights $\boldsymbol{\alpha}$—through genetic algorithm operations such as selection, crossover, and mutation—toward configurations that yield toolsets with higher fitness.

2.3 DYNAMIC ENGA

Objective

$$\tilde{\Delta}(v|X_j, s_t) = \alpha_1 \cdot \mathcal{I}_{\text{rel}}(v, q, s_t) + \alpha_2 \cdot \mathcal{S}_{\text{syn}}(v, \mathcal{X}_j) - \alpha_3 \cdot \mathcal{C}_{\text{run}}(v)$$

离线训阿尔法，协同值，消耗值。
I存向量库，query embedding后匹配得分
在线变动：I随query变，S随候选工具集变化

1. α_1: State-Goal Alignment

In a web environment, the relevance of a tool depends not only on the global goal q (e.g., "book an early flight to New York tomorrow") but also—and more critically—on the current page state s_t (e.g., currently on the "passenger information" page).

Physical Significance: The local utility of a candidate action v in advancing the global task q, given the current DOM state.

Calculation Method: Extract the text and attributes of the UI element associated with candidate action v (e.g., '<button aria-label="Search Flights">').

Use a lightweight encoder to compute the element's embedding and calculate its similarity to the current sub-task intent (derived from the goal q and the current state s_t).

Scenario Mapping: If the current state s_t is the homepage and the goal is to book a flight, the action $v =$ type[Input_Destination, "JFK"] would receive a very high score, whereas $v =$ click[FAQ_Link] would receive a very low score.

2. α_2: Sequential Dependency & Coherence

Actions in WebArena are governed by strict causal relationships—specifically, "pre-conditions" and "post-conditions." For instance, one cannot click "Search" before selecting a date, nor click "Pay" without first agreeing to the terms and conditions.

Physical Significance: The logical coherence between the candidate tool v and the sequence of actions already performed ($\mathcal{X}_j$); specifically, whether the state resulting from $\mathcal{X}_j$ satisfies the pre-conditions required for v.

Calculation Method: Schema-based Pre/Post-condition Matching: For example, the pre-condition for the action click[Search] typically requires that specific input fields have already been populated with values. By defining or learning a transition matrix, one can calculate the conditional transition probability $P(v|\mathcal{X}_j)$ of an action v given a historical trajectory $\mathcal{X}_j$.

DOM Focus Tracking: Assess whether the UI element acted upon by v maintains reasonable continuity—either in physical space or logical hierarchy—with the element just interacted with in $\mathcal{X}_j$ (e.g., filling in the "First Name" field immediately after the "Last Name" field).

Scenario Mapping: If $\mathcal{X}_j$ includes click[Departure_Date_Input], then the subsequent candidate action $v =$ click[Calendar_Day_24] receives a significant synergy bonus.

3. α_3: Execution Risk & Irreversibility

In web navigation, the costs associated with executing incorrect actions are asymmetric. The cost of scrolling down a page is minimal (as it can be reversed by scrolling up), whereas clicking "Submit Order" or exiting the current booking flow carries a very high cost and could lead to the failure of the entire episode. Furthermore, web pages are often extremely lengthy, and processing complex DOM trees consumes a large number of tokens.

Physical Significance: The execution cost of an action (token consumption) and the potential risk of causing a state collapse.

Calculation Method: Token / Time Cost: Estimate the DOM complexity of the next state that the system must parse after executing action v.

Irreversibility Penalty: Assign a penalty coefficient to high-risk actions. For instance, form submission, payment, and cross-domain navigation are classified as high-risk actions, whereas hovering and scrolling are low-risk. For high-risk actions, the algorithm adopts a more conservative approach when evaluating their Δ value (i.e., they require strong positive support from α_1 and α_2 to be selected).

Scenario Mapping: For $v =$ click[Confirm_Payment], the value of $\mathcal{C}_{\text{risk}}$ would be high. Unless $\mathcal{X}_j$ already perfectly encapsulates all necessary steps—such as seat selection and form filling—(resulting in an extremely high $\mathcal{S}_{\text{seq}}$), the algorithm will not greedily select it during the early stages of exploration.

Algorithm 3 Dynamic Interactive Nested Greedy Algorithm (DIN-GA) in WebArena

Input: WebArena Environment $\mathcal{E}$; Goal query q; Horizon budget T; Weight vector $\boldsymbol{\alpha} = [\alpha_1, \alpha_2, \alpha_3]$
Output: Execution trajectory $\mathcal{X}_T$ and task success status

1: Initialize WebArena State $s_1 \leftarrow \mathcal{E}.\text{reset}(q)$
2: Initialize historical action trajectory $\mathcal{X}_0 \leftarrow \emptyset$
3: **for** step $t = 1$ ... T **do**
4: Parse current DOM/Accessibility tree from s_t to extract valid action candidates $\mathcal{V}(s_t)$
5: **if** s_t is terminal or $\mathcal{V}(s_t) = \emptyset$ **then**
6: **break** ▷ Episode finished or task deadlocked
7: **end if**
8: **// Phase 1: State-Conditioned Marginal Gain Evaluation**
9: **for** each candidate action $v \in \mathcal{V}(s_t)$ **do**
10: Compute State-Goal Alignment: $\mathcal{I}_{\text{rel}}(v, q, s_t)$
11: Compute Sequential Dependency Score: $\mathcal{S}_{\text{seq}}(v, \mathcal{X}_{<t})$
12: Compute Execution Risk & Cost: $\mathcal{C}_{\text{risk}}(v)$
13: **// Calculate parameterized dynamic marginal gain $\tilde{\Delta}$**
14: $\tilde{\Delta}(v \mid \mathcal{X}_{<t}, s_t) \leftarrow \alpha_1 \cdot \mathcal{I}_{\text{rel}}(v, q, s_t) + \alpha_2 \cdot \mathcal{S}_{\text{seq}}(v, \mathcal{X}_{<t}) - \alpha_3 \cdot \mathcal{C}_{\text{risk}}(v)$
15: **end for**
16: **// Phase 2: Action Selection**
17: $v_t^* \leftarrow \arg\max_{v \in \mathcal{V}(s_t)} \tilde{\Delta}(v \mid \mathcal{X}_{<t}, s_t)$
18: **// Phase 3: Real Environment Interaction & State Transition**
19: Execute action v_t^* in WebArena: $(s_{t+1}, r_t, \text{done}) \leftarrow \mathcal{E}.\text{step}(v_t^*)$
20: Update action trajectory: $\mathcal{X}_t \leftarrow \mathcal{X}_{<t} \circ (s_t, v_t^*)$
21: **if** done **then**
22: **break** ▷ Goal achieved or failure triggered
23: **end if**
24: **end for**
return Trajectory $\mathcal{X}_t$ and final environment task status

Explanation Line 4 ('GetValidActions'): In web navigation, the available tools are not static. At each step, the algorithm dynamically extracts interactive elements (such as text inputs, clickable buttons, and dropdown menus) by parsing the current page (DOM tree) to form the action pool $\mathcal{V}(s_t)$ for that step.

Lines 9–13 (Calculation of $\tilde{\Delta}$): $\mathcal{I}_{\text{rel}}$: Evaluates whether action v aligns with the global booking goal q given the current page state s_t (e.g., on a flight list page, selecting "sort by price" or "choose early flight" yields a higher alignment score).

$\mathcal{S}_{\text{seq}}$: Evaluates the coherence of action v with the historical trajectory $\mathcal{X}_{<t}$ of the preceding $t-1$ steps (e.g., entering an ID number immediately after entering a passenger's name).

$\mathcal{C}_{\text{risk}}$: Applies a penalty to irreversible or high-cost actions—such as "submit payment" or "cancel order"—to discourage blind exploration.

Line 18 ('Environment Execution'): Executes the selected action v_t^* within the WebArena simulator and obtains the new state s_{t+1} following the resulting webpage refresh, thereby establishing a dynamic closed-loop process.

REFERENCES

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024. URL https://arxiv.org/abs/2307.13854.